Convertir grados Celsius a grados Fahrenheit

Código del ejemplo explicado de TensorFlow

Crear un modelo que nos permitirá convertir grados celsius a grados Fahrenheit sin utilizar una fórmula. Le proporcionaremos al modelo una serie de unos 30 datos de ejemplos de conversión y con base en ellos, el modelo “aprenderá” cómo convertir de grados celsius a grados fahrenheit.

Paso 1. Importar las librerías

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
```

Las librerías que utilizaremos son las que mismas de los modelos de regresión e incluimos Tensorflow para crear un modelo de machine learning.

Paso 2. Obtención de los datos

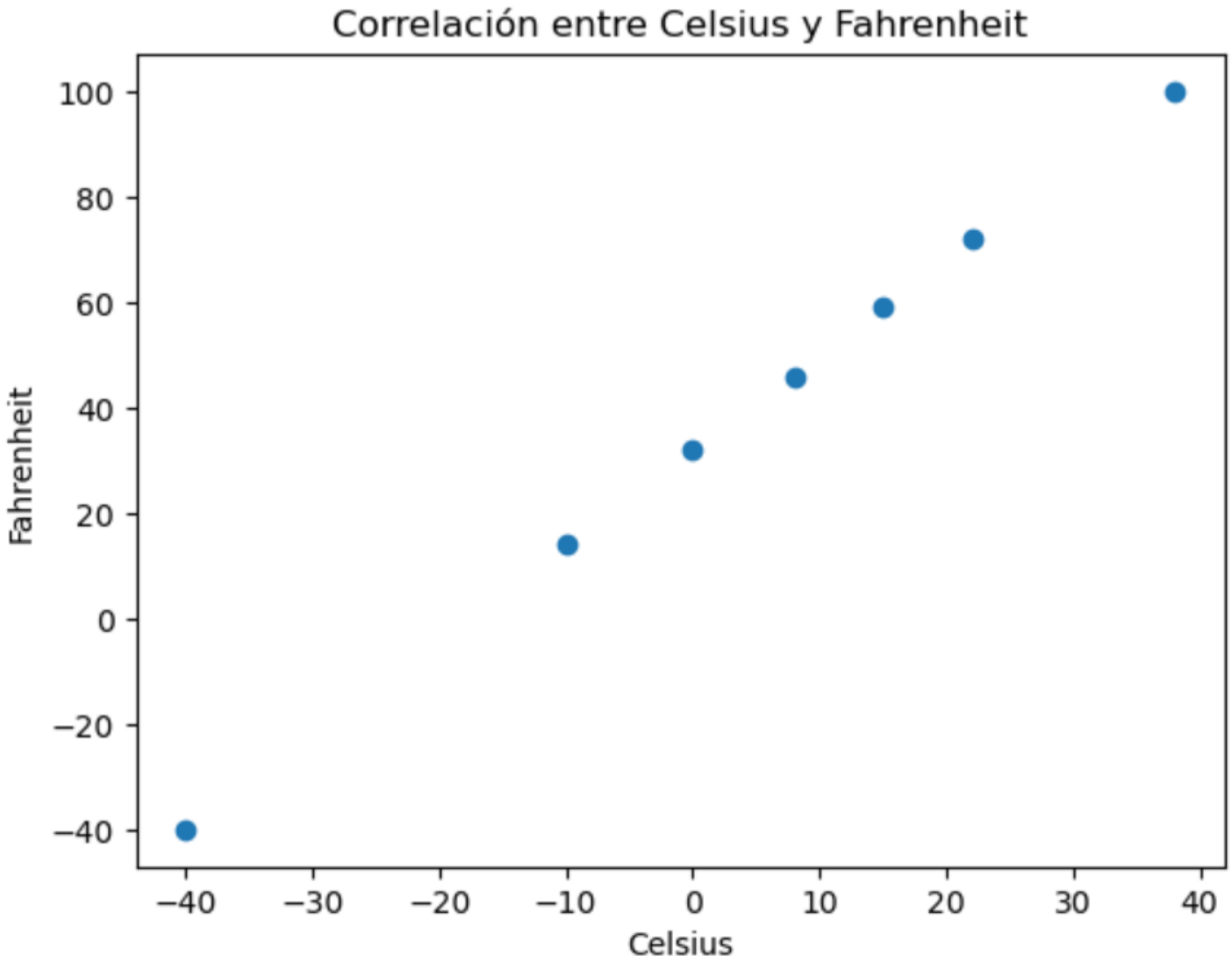
```
celsius = np.array([-40, -10, 0, 8, 15, 22, 38], dtype=float)
fahrenheit = np.array([-40, 14, 32, 46, 59, 72, 100], dtype=float)
```

Primero, leemos los datos del arreglo y mostramos su correlación.

```
plt.scatter(celsius, fahrenheit)

plt.title('Correlación entre Celsius y Fahrenheit')
plt.xlabel('Celsius')
plt.ylabel('Fahrenheit')

plt.show()
```



Paso 3. Selección de las variables independiente y objetivo

```
X_train = celsius
Y_train = fahrenheit
```

Los datos obtenidos contienen dos columnas. La primer llamada Celsius, será nuestra variable independiente. La segunda, llamada Fahrenheit, será la variable objetivo. Es decir, una vez entrenado, el modelo será capaz de indicar los grados fahrenheit si le proporcionamos los grados celsius.

Paso 4. Creación del modelo

```
# Paso 4. Creación del modelo
print("Creando el modelo")
model = tf.keras.Sequential()
model.add(tf.keras.layers.Input(shape=(1,)))
model.add(tf.keras.layers.Dense(units=1))
```

En ésta parte del código utilizamos Keras. Keras es una biblioteca escrita en Python que nos permite crear redes neuronales. Keras nos permite acceder a varios frameworks de machine learning, entre ellos, TensorFlow.

keras.Sequential nos permite crear una red neuronal en keras a la que se le agregan “capas” en forma secuencial. Un modelo en keras se considera “como una secuencia de capas que cada una de ellas va “destilando” gradualmente los datos de entrada para obtener la salida deseada” (Torres, 2018).

add agrega una capa al modelo. Cuando un modelo tiene capas densamente conectadas, todas las neuronas de cada capa están conectadas con todas las neuronas de la capa siguiente.

Paso 5. Compilación del modelo

```
# Paso 5. Compilación del modelo
print("Compilando el modelo")
model.compile(optimizer=tf.keras.optimizers.Adam(1.0), loss='mean_squared_error')
```

Antes de que el modelo sea entrenado, se agregan configuraciones adicionales durante el paso de compilación. En éste caso se configura un optimizador y una función de pérdida, que mide qué tan exacto es el modelo. La función de pérdida utiliza es el “error cuadrado promedio” que básicamente mide el error cuadrado promedio de nuestras predicciones. El modelo tratará de minimizar ésta función para dirigir el modelo a la dirección adecuada.

Paso 6. Entrenamiento del modelo

```
# Paso 6. Entrenamiento del modelo
print("Entrenando el modelo")
epochs_hist = model.fit(X_train, y_train, epochs=100)
```

Usamos **fit** para entrenar el modelo. Le pasamos como parámetros la variable independiente (Grados Celsius) y la variable objetivo (Grados Fahrenheit) . También le indicamos que deseamos 100 epochs. Un **epoch** representa una iteración completa del conjunto de entrenamiento. El número de epochs es la cantidad de veces que se ejecutará el algoritmo.

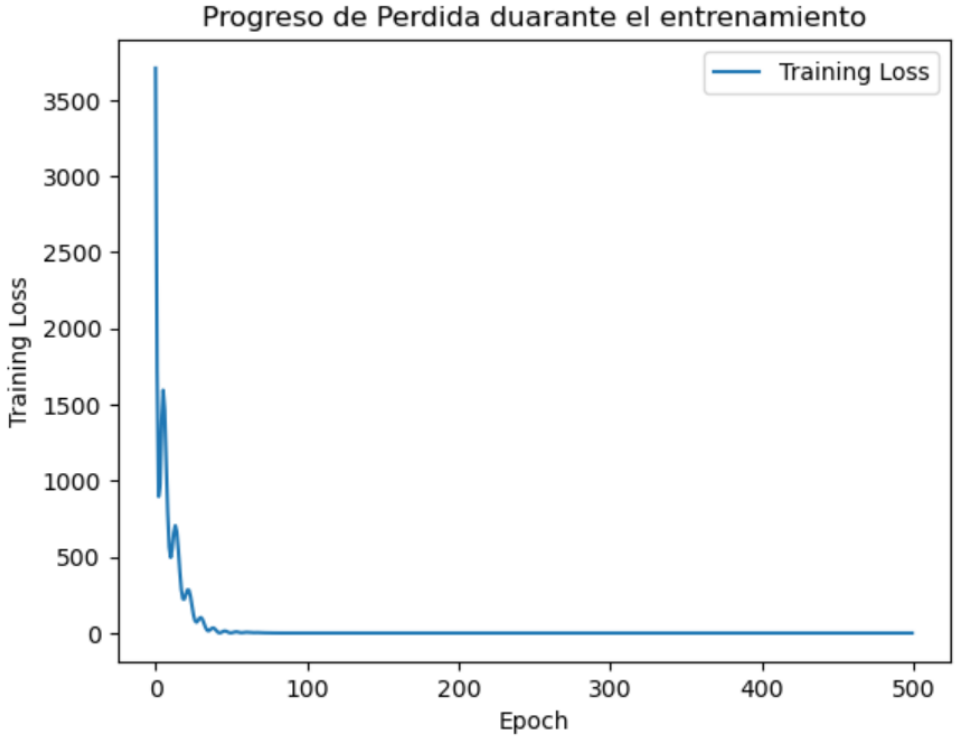
Puedes agregar el parámetro **`verbose=False`** en la función fit para que no muestre todas las épocas.

```
# Evaluación del modelo
print("Evaluando el modelo entrenado")
print("Keys:")
print(epochs_hist.history.keys())

# Gráfico
plt.plot(epochs_hist.history['loss'])
plt.title('Progreso de Pérdida durante Entrenamiento del Modelo')
plt.xlabel('Epoch')
plt.ylabel('Training Loss')
plt.legend(['Training Loss'])
plt.show()
```

A continuación, evaluaremos el modelo.

Primero utilizamos epochs_hist.history.keys() para mostrar el nombre de las llaves. Observamos que el nombre es “loss”. Después, procedemos a graficar ese dato.



Observamos que la pérdida (Training_loss) va disminuyendo a medida que se realizan más iteraciones (epoch).
Realizar una predicción utilizando el modelo entrenado

```
Temp_C = 100
Temp_C_array = np.array([[Temp_C]])
Temp_F = model.predict(Temp_C_array)

print("Temperatura de Prediccion: " + str(Temp_F))
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 59ms/step
Temperatura de Prediccion: [[211.7474]]
```

Teniendo nuestro modelo entrenado, ya podemos realizar una predicción. En éste caso, le damos una valor en grados Celsius (Temp_C) y el modelo nos devuelve el resultado que el cree correcto en grados Fahrenheit (Temp_F).

Adicionalmente, mostramos el resultado utilizando la fórmula matemática, para corroborar qué tan cercana fue la predicción que el modelo arrojó.

```
Temp_F = 9/5 * Temp_C + 32
print("Temperatura de Ecuacion: " + str(Temp_F))
```

```
Temperatura de Ecuacion: 212.0
```

Con éste ejemplo se inicia con el conocimiento sobre la biblioteca para Machine Learning que es TensorFlow.

Modificaciones

1. Utilizar la librería celsius_a_fahrenheit.csv de seaborn para entrenar el modelo.
2. Mostrar los primeros 5 datos
3. Para mostrar las correlación de los datos de seaborn se utiliza la función scatterplot.
   `sns.scatterplot(x='Celsius', y='Fahrenheit', data=df_temperatura)`
4. Probar con las misma cantidad de capas, revisar los resultados y comparar las graficas de pérdidas.
5. Agregar una o dos capas más, , revisar los resultados y comparar las graficas de pérdidas.
6. Puedes optar por almacenar las capas de la red neuronal en variables, para después mostrar los valores utilizados por el modelo para su entrenamiento

```
oculta1 = tf.keras.layers.Dense(units=3, input_shape=[1])
oculta2 = tf.keras.layers.Dense(units=3)
salida = tf.keras.layers.Dense(units=1)
modelo = tf.keras.Sequential([oculta1, oculta2, salida])
```

```
print(oculta1.get_weights())
print(oculta2.get_weights())
print(salida.get_weights())
```